INF461 · PRINCIPES ET PROCESSUS DU LOGICIEL

Le patron Abstract Factory

Exemple du cours et variante concrète : familles de composants d'interface selon le thème

Étudiant : Theodore Wilfried Bitjong

Matricule : 23U2331

Dépôt GitHub :
https://github.com/inf461-bitjong/abstact-factory

11 novembre 2026

1. Présentation du patron

Abstract Factory (Fabrique abstraite) est un patron de création qui fournit une interface pour créer plusieurs produits liés, sans exposer leurs classes concrètes. Le client choisit une fabrique concrète et utilise les produits retournés au travers de leurs abstractions.

Structure générique

- **Fabrique abstraite :** déclare une opération de création pour chaque type de produit de la famille.
- **Fabriques concrètes :** créent les produits concrets qui composent chacune des familles.
- **Produits abstraits et concrets :** définissent les interfaces communes et leurs variantes.
- **Client :** dépend des abstractions et emploie ensemble les produits d'une même fabrique.

> **Choix du modèle générique.** Le besoin porte sur la création cohérente de plusieurs types de produits. Dans l'exemple du cours, chaque fabrique produit un ProduitA et un ProduitB appartenant à la même famille. Dans la variante, chaque fabrique de thème crée un bouton et un menu assortis. Abstract Factory évite au client de connaître les classes concrètes et facilite l'ajout d'une famille complète.

Intérêt et limite

Le changement de famille se fait en choisissant une autre fabrique concrète, tandis que le code client reste fondé sur les mêmes abstractions. En contrepartie, l'ajout d'un nouveau type de produit nécessite de modifier l'interface de fabrique et chacune de ses implémentations.

2. Exemple vu en cours

L'exemple présente deux familles de produits. `ProduitFactory1` crée `ProduitA1` et `ProduitB1`, tandis que `ProduitFactory2` crée `ProduitA2` et `ProduitB2`. Le client appelle les méthodes des abstractions sans dépendre des implémentations concrètes.

Modélisation

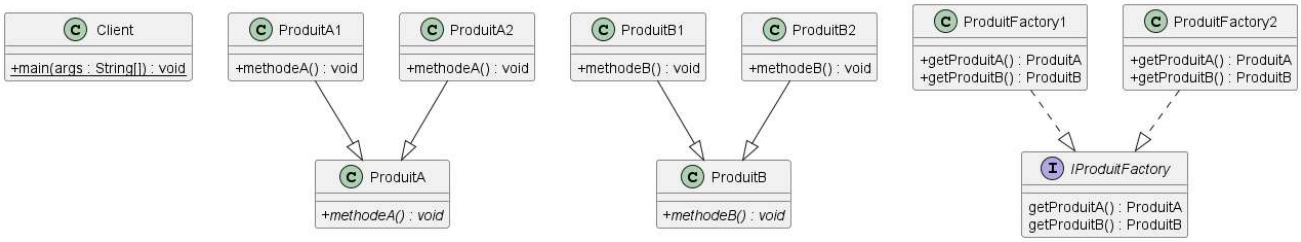


Figure 1 — Diagramme de classe de l'exemple du cours.

> **Lecture du diagramme :** les deux fabriques concrètes implémentent `IProduitFactory`. Elles créent chacune un produit concret de type A et un produit concret de type B. Les classes concrètes héritent de leurs produits abstraits respectifs.

Comportement du client

`Client` instancie successivement les deux fabriques, récupère les deux produits de chaque famille, puis appelle `methodeA()` et `methodeB()`. La sortie montre les implémentations qui ont été sélectionnées.

3. Implémentation de l'exemple du cours

Abstractions — IProduitFactory.java, ProduitA.java et ProduitB.java

```
package cours.abstractions;

public interface IProduitFactory {
    ProduitA getProduitA();
    ProduitB getProduitB();
}

public abstract class ProduitA {
    public abstract void methodeA();
}

public abstract class ProduitB {
    public abstract void methodeB();
}
```

Fabriques — ProduitFactory1.java et ProduitFactory2.java

```
package cours.factories;

public class ProduitFactory1 implements IProduitFactory {
    @Override
    public ProduitA getProduitA() {
        return new ProduitA1();
    }

    @Override
    public ProduitB getProduitB() {
        return new ProduitB1();
    }
}

public class ProduitFactory2 implements IProduitFactory {
    @Override
    public ProduitA getProduitA() {
        return new ProduitA2();
    }

    @Override
    public ProduitB getProduitB() {
        return new ProduitB2();
    }
}
```

Les imports des classes de chaque fichier sont omis dans ces extraits ; les méthodes et relations montrées correspondent au code source du dépôt.

3. Implémentation de l'exemple du cours

Produits concrets — modèles A et B

```
package cours.models;
import cours.abstractions.ProduitA;

public class ProduitA1 extends ProduitA {
    @Override
    public void methodeA() {
        System.out.println("ProduitA1.methodeA()");
    }
}

public class ProduitA2 extends ProduitA {
    @Override
    public void methodeA() {
        System.out.println("ProduitA2.methodeA()");
    }
}

package cours.models;
import cours.abstractions.ProduitB;

public class ProduitB1 extends ProduitB {
    @Override
    public void methodeB() {
        System.out.println("ProduitB1.methodeB()");
    }
}

public class ProduitB2 extends ProduitB {
    @Override
    public void methodeB() {
        System.out.println("ProduitB2.methodeB()");
    }
}
```

Client.java

```
package cours;

public class Client {
    public static void main(String[] args) {
        IProduitFactory produitFactory1 = new ProduitFactory1();
        IProduitFactory produitFactory2 = new ProduitFactory2();

        System.out.println("--- Utilisation de la premiere fabrique (Famille 1) ---");
        ProduitA produitA = produitFactory1.getProduitA();
        ProduitB produitB = produitFactory1.getProduitB();
        produitA.methodeA();
        produitB.methodeB();

        System.out.println("\n--- Utilisation de la seconde fabrique (Famille 2) ---");
        produitA = produitFactory2.getProduitA();
        produitB = produitFactory2.getProduitB();
        produitA.methodeA();
        produitB.methodeB();
    }
}
```

Les imports des classes utilisées dans les fichiers sont omis uniquement pour condenser la présentation.

4. Exemple modifié : thèmes d'interface

La variante applique le patron à deux thèmes d'interface, sombre et clair. Chaque thème fournit une implémentation de `Bouton` et de `Menu`. `ClientVariante` construit les deux familles et invoque leurs opérations d'affichage.

Modélisation

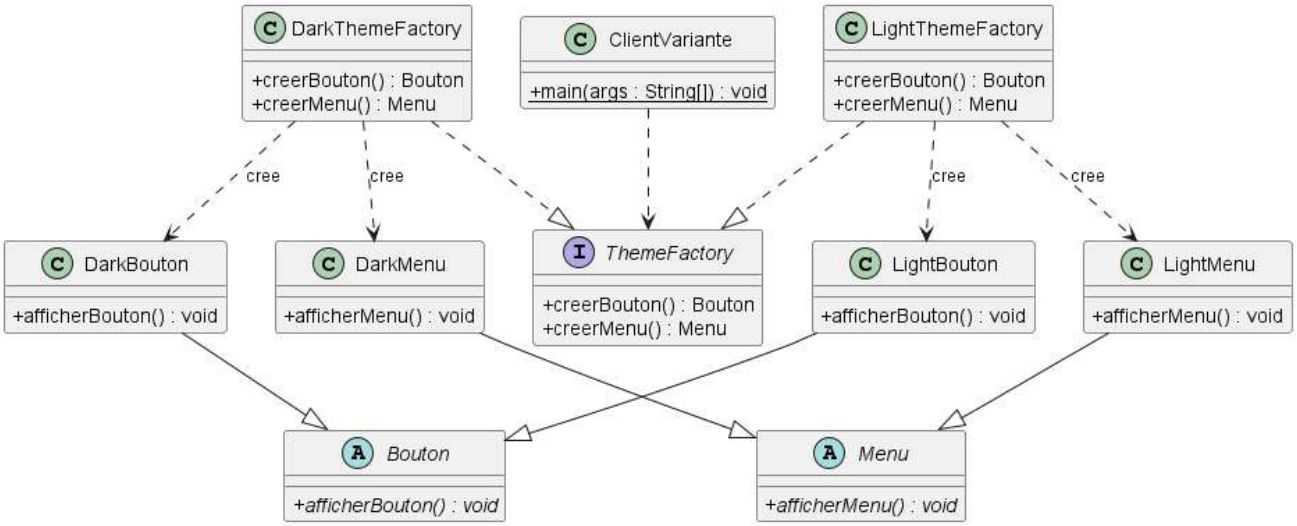


Figure 2 — Diagramme de classe des fabriques de thèmes et des composants d'interface.

Analyse du choix

Un thème est une famille de composants qui doivent présenter un style cohérent. `DarkThemeFactory` associe `DarkBouton` à `DarkMenu` et `LightThemeFactory` associe les composants clairs. Le client dépend de `ThemeFactory`, `Bouton` et `Menu` plutôt que des classes concrètes ; il peut donc changer de famille en substituant la fabrique.

> **Résultat attendu :** le programme affiche les messages du bouton et du menu du thème sombre, puis ceux du thème clair. Cette variante illustre la création de produits assortis ; le rendu est ici simulé par des messages dans la console.

5. Implémentation de la variante

Abstractions et fabrique — ThemeFactory, Bouton et Menu

```
package variante.abstractions;

public interface ThemeFactory {
    Bouton creerBouton();
    Menu creerMenu();
}

public abstract class Bouton {
    public abstract void afficherBouton();
}

public abstract class Menu {
    public abstract void afficherMenu();
}
```

Fabriques concrètes — DarkThemeFactory et LightThemeFactory

```
package variante.factories;

public class DarkThemeFactory implements ThemeFactory {
    @Override
    public Bouton creerBouton() {
        return new DarkBouton();
    }

    @Override
    public Menu creerMenu() {
        return new DarkMenu();
    }
}

public class LightThemeFactory implements ThemeFactory {
    @Override
    public Bouton creerBouton() {
        return new LightBouton();
    }

    @Override
    public Menu creerMenu() {
        return new LightMenu();
    }
}
```

Les imports de chaque fichier sont omis ; les extraits regroupent les types par rôle pour faciliter la lecture.

5. Implémentation de la variante

Produits concrets — boutons et menus

```
package variante.models;

public class DarkBouton extends Bouton {
    @Override
    public void afficherBouton() {
        System.out.println("[DARK MODE] Affichage d'un bouton sombre (fond gris fonce, texte blanc).");
    }
}

public class DarkMenu extends Menu {
    @Override
    public void afficherMenu() {
        System.out.println("[DARK MODE] Affichage d'un menu deroulant sombre.");
    }
}

public class LightBouton extends Bouton {
    @Override
    public void afficherBouton() {
        System.out.println("[LIGHT MODE] Affichage d'un bouton lumineux (fond blanc, texte noir).");
    }
}

public class LightMenu extends Menu {
    @Override
    public void afficherMenu() {
        System.out.println("[LIGHT MODE] Affichage d'un menu deroulant lumineux.");
    }
}
```

ClientVariante.java

```
package variante;

public class ClientVariante {
    public static void main(String[] args) {
        System.out.println("=== EXEMPLE MODIFIE : Abstract Factory (Themes UI) ===");

        ThemeFactory darkFactory = new DarkThemeFactory();
        Bouton darkBouton = darkFactory.creerBouton();
        Menu darkMenu = darkFactory.creerMenu();
        System.out.println("\nChargement du theme sombre :");
        darkBouton.afficherBouton();
        darkMenu.afficherMenu();

        ThemeFactory lightFactory = new LightThemeFactory();
        Bouton lightBouton = lightFactory.creerBouton();
        Menu lightMenu = lightFactory.creerMenu();
        System.out.println("\nChargement du theme clair :");
        lightBouton.afficherBouton();
        lightMenu.afficherMenu();
    }
}
```

Les imports de chaque fichier sont omis ; les méthodes et messages affichés sont ceux du code source du dépôt.